/loop

Goal-driven agent loop: /loop opens setup, /loop starts, /loop help shows usage

Overview

`/loop` runs a goal-driven agentic loop that drives the selected agent toward an explicit goal until a stop condition is reached. A run stops when the goal is achieved, the verification gate decides the outcome, the step or cost budget is exhausted, or the run is interrupted.

The command has two surfaces: the interactive setup dialog (opened by bare `/loop`) and the one-shot command form that accepts an agent, optional flags, and the goal in a single line. Flags may appear in any position and accept both `--flag value` and `--flag=value` spellings.

Syntax

```
/loop
/loop help
/loop <agent> <goal>
/loop <agent> [--max-steps N] [--cost_limit N] [--timeout N] <goal>
```

Flag rules:

- Flags may appear anywhere in the line, before or after the goal text.
- `--flag value` and `--flag=value` are both accepted.
- `--cost_limit` and `--cost-limit` are both accepted spellings.
- Values must be non-negative integers.

Options

Form	Description
`/loop` (bare)	Opens the interactive setup dialog; no run is started.
`help`	Prints the usage help.
`<agent>`	Agent preset that runs the loop (e.g. `coder`, `general`).
`<goal>`	The goal text the loop works toward. Required.
`--max-steps N`	Step budget override for this run (non-negative integer).
`--cost_limit N`	Cost budget override (alias `--cost-limit`).
`--timeout N`	Run timeout in seconds (non-negative integer).

Defaults when flags are omitted:

Setting	Default
Max steps	`loop.max_steps` from config (512)
Cost limit	No cost limit
Checkpoints	Enabled

Setup dialog

Bare `/loop` opens the setup dialog with these fields:

Field	Purpose
Agent	Agent preset used for the run
Goal	The goal text the loop works toward
Verify cmd	Optional verification command run by the verification gate
Scope	Scope globs restricting where the loop may operate
Read-only	Read-only globs the loop may read but not modify
Tool set	Restricts the tool set available to the loop
Max steps	Step budget override
Cost limit	Cost budget override
Checkpoints	Toggles pre-loop snapshot capture on or off

Dialog keys:

Key	Action
Tab / Shift+Tab	Move between fields
Up / Down	Move between fields
Space	Toggle the Checkpoints field
Enter	Confirm and start the loop

If the goal is missing when a run is requested, the loop reports an error and re-opens the dialog with the agent pre-selected.

Examples

```
/loop
```

Opens the setup dialog.

```
/loop coder Fix the failing unit tests in crates/ragent-storage
```

```
/loop coder --max-steps 64 Refactor parse_flags into smaller functions
```

```
/loop coder --max-steps=100 --cost_limit=5 --timeout=1800 Make cargo test green
```

```
/loop debug --cost-limit 2 Diagnose the intermittent TUI scroll panic
```

```
/loop help
```

Output

- On start the loop reports the goal, agent, and any active overrides, then runs step-by-step with the budget enforced.
- Stop conditions: `completed`, `error`, `budget_exhausted`, or `interrupted`.
- With checkpoints enabled, a pre-loop snapshot is captured before the first step; after the loop the rollback flow offers restoring it (Enter restores the pre-loop snapshot, Esc keeps the changes).

- With a verify command configured, the verification gate runs after the loop and its result participates in the stop decision.

Configuration

Config key	Default	Purpose
`loop.max_steps`	512	Default step budget when no `--max-steps` flag is given
`loop.cost_limit`	none	Default cost budget when no `--cost_limit` flag is given
`loop.checkpoint_timeout_secs`	120	Checkpoint timeout in seconds

Related

- `/autopilot` – autonomous operation with auto-approved permissions
- `/goal` – set a goal condition evaluated after each turn
- `/resume` – continue a halted agent run
- `/undo` – snapshot rollback for file edits
- `ragent.json` `loop` section – loop defaults and checkpoint timeout